

Sorting Algorithms: A Quick-Reference Guide

A compact study sheet covering comparison-based and non-comparison sorting methods, built for fast revision before an exam or viva.

Summary Table

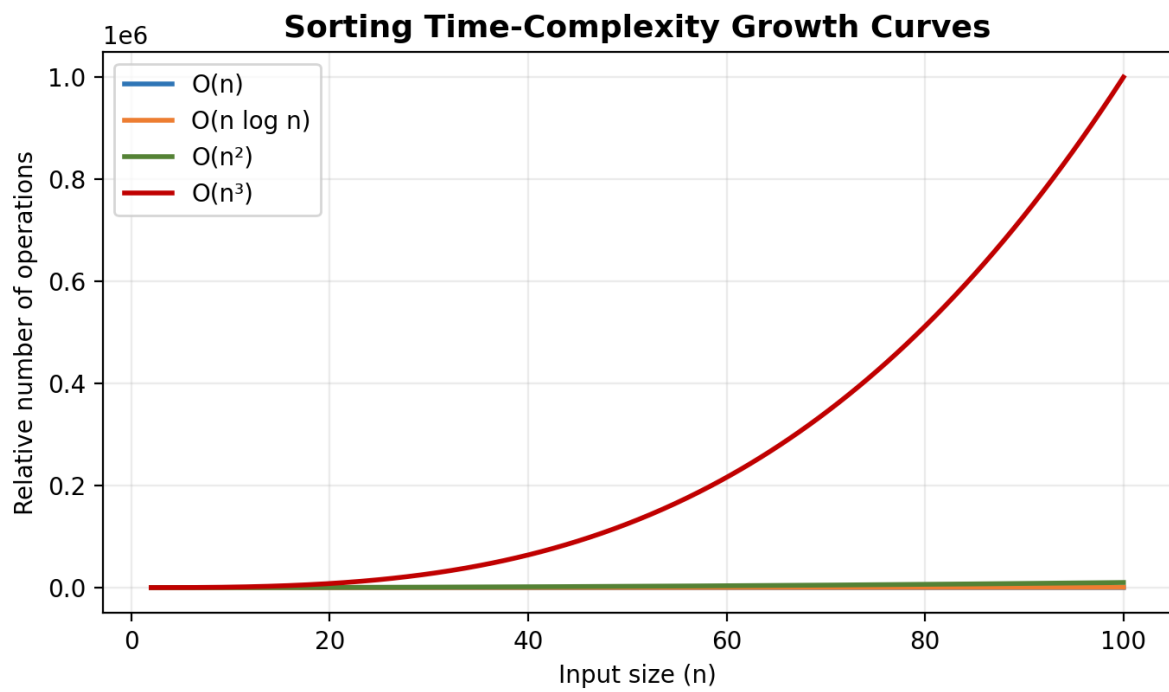
The table below lines up every algorithm against its running time in the best, average and worst case, along with memory use and two key properties: whether equal elements keep their original order (stability) and whether the algorithm sorts within the original array (in-place).

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No*	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No**
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^{***}$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Shell Sort	Varies	$\approx O(n^{1.5})^{****}$	$O(n^2)^{****}$	$O(1)$	No	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes**** *	No
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes**** *	No
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	Yes**** **	Usually no

* A textbook selection sort does not preserve order among equal keys. ** Classic array-based merge sort needs $O(n)$ extra memory. *** The recursion stack averages $O(\log n)$ but can grow to $O(n)$ in the worst case. **** Shell sort's actual bound shifts with the gap sequence chosen. ***** These reach stability only when a stable placement step is used. ***** Whether bucket sort is stable depends on the sort used inside each bucket.

Visualizing Growth Rates

The chart plots the four complexity classes that show up again and again in sorting analysis. Note that $O(n+k)$ and $O(d(n+k))$ aren't included here, since they scale with the key range or digit count rather than n alone, so they don't follow one fixed curve.



Algorithm-by-Algorithm Notes

1. Bubble Sort

Walks through the array comparing neighboring pairs and swapping them if they're out of order. Each full pass pushes the largest remaining value into its final spot.

- Best: $O(n)$ — achievable with a flag that stops early once a pass makes no swaps.
- Average: $O(n^2)$.
- Worst: $O(n^2)$, as with a fully reverse-sorted list.
- Space: $O(1)$; stable; sorts in place.

Note: Mostly useful for teaching or for data that's already almost sorted — it doesn't scale well beyond that.

2. Selection Sort

Scans the unsorted remainder for the smallest value and swaps it into the next open slot at the front.

- Best, average, and worst: $O(n^2)$ comparisons.
- Space: $O(1)$; sorts in place.
- Not stable in its standard form.

Note: Its selling point is doing only $O(n)$ swaps total, which helps when each write is costly.

3. Insertion Sort

Grows a sorted section at the front of the array one item at a time, sliding larger values over to make room for the new one.

- Best: $O(n)$ on data that's already sorted or nearly so.
- Average and worst: $O(n^2)$.
- Space: $O(1)$; stable; sorts in place.

Note: A strong choice for small or nearly-sorted arrays, and it's frequently embedded inside hybrid sorting algorithms.

4. Merge Sort

A divide-and-conquer method: split the array in half, sort each half recursively, then merge the two sorted halves back together.

- Best, average, and worst: $O(n \log n)$.
- Space: $O(n)$ for the typical array-based version.
- Stable, but not in-place in the standard implementation.

Note: Its worst case is as good as its best case, and it's a natural pick whenever stability matters.

5. Quick Sort

Picks a pivot value, partitions the rest of the array around it, then recurses on the two resulting parts.

- Best and average: $O(n \log n)$.

- Worst: $O(n^2)$ when pivot choices keep being poor.
- Recursion space: $O(\log n)$ on average, $O(n)$ in the worst case.
- Sorts in place in common implementations; not stable.

Note: Fast in practice, and using a randomized or well-chosen pivot makes the bad worst case unlikely.

6. Heap Sort

Builds a binary heap from the data, then repeatedly pulls out the largest (or smallest) element to place it in its final position.

- Best, average, and worst: $O(n \log n)$.
- Space: $O(1)$ extra for the array-based version.
- Sorts in place; not stable.

Note: A good fit when you need a guaranteed $O(n \log n)$ ceiling without paying for extra memory.

7. Shell Sort

An extension of insertion sort that first compares elements spaced far apart using a gap sequence, shrinking the gap on each pass until it reaches 1.

- Running time depends heavily on which gap sequence is used.
- Commonly cited worst case: around $O(n^2)$.
- Space: $O(1)$; sorts in place; not stable.

Note: Often beats plain insertion sort on mid-sized arrays, though its behavior is less predictable than true $O(n \log n)$ methods.

8. Counting Sort

Tallies how often each distinct key value appears, then uses running totals of those counts to place every element directly into its sorted position.

- Time: $O(n+k)$, with k being the spread of key values.
- Space: $O(n+k)$ in the standard stable version.
- Not comparison-based; can be made stable.

Note: Shines when the key range k stays close to n and the keys are integers within a workable range.

9. Radix Sort

Sorts keys one digit position at a time — typically starting from the least significant digit — using a stable helper routine such as counting sort at each pass.

- Time: $O(d(n+k))$, where d is the digit/pass count and k the range per digit.
- Extra space: typically $O(n+k)$.
- Not comparison-based; can be made stable.

Note: A good match for integers, fixed-length strings, or any key that breaks cleanly into digits.

10. Bucket Sort

Spreads elements across a set of buckets based on value range, sorts the contents of each bucket, then concatenates the buckets in order.

- Best and average: $O(n+k)$ when values are reasonably well spread out.
- Worst: $O(n^2)$ if too many elements land in one bucket and its internal sort is quadratic.
- Extra space: $O(n+k)$; stability depends on the method used inside buckets.

Note: Performs well when the input values are roughly uniform across a known range.

Picking the Right Algorithm

A quick lookup for matching a situation to a sensible default choice.

Situation	Good Choice	Why
Small or nearly-sorted input	Insertion Sort	Very quick when only a few elements need to move.
Need a guaranteed $O(n \log n)$	Merge Sort / Heap Sort	Neither one ever falls into quadratic time.
General-purpose, fast in practice	Quick Sort / hybrid variants	Strong average speed and good cache behavior.
Stability is required	Merge Sort / stable Insertion Sort	Equal-key elements keep their original order.
Integer keys, small range	Counting Sort	Can hit $O(n+k)$ time.
Integer or fixed-digit keys	Radix Sort	Sidesteps the comparison-sort lower bound.
Values spread evenly over a range	Bucket Sort	Can approach linear expected time.
Extra memory is tight	Heap Sort / Insertion Sort	Both use $O(1)$ auxiliary space in their usual form.

Points Worth Remembering

- Any comparison-based sort is bound below by $\Omega(n \log n)$ comparisons in the worst case, for arbitrary input.
- Bubble, selection, and insertion sort all run in $O(n^2)$ on average and in the worst case — though insertion sort and a flag-optimized bubble sort drop to $O(n)$ on data that's already sorted.
- Merge sort's $O(n \log n)$ time holds in every case and it's stable, but the standard array version costs extra memory.
- Quick sort averages $O(n \log n)$ but can degrade to $O(n^2)$ with unlucky pivot choices.
- Heap sort guarantees $O(n \log n)$ in the worst case while using only $O(1)$ extra array space.
- Counting sort, radix sort, and bucket sort skip comparisons entirely, letting them beat the comparison lower bound whenever their assumptions about the input hold.